



# Data Engineering

## CAPSTONE PROJECT



End-to-End Modern Data Pipeline



## Capstone Project

### Build an End-to-End Modern Data Engineering Pipeline

**Level:** Beginner → Intermediate Data Engineering

**Technology Stack:** Python • Fake Store API • Snowflake • dbt • Apache Airflow • Power BI • GitHub • GitHub Actions

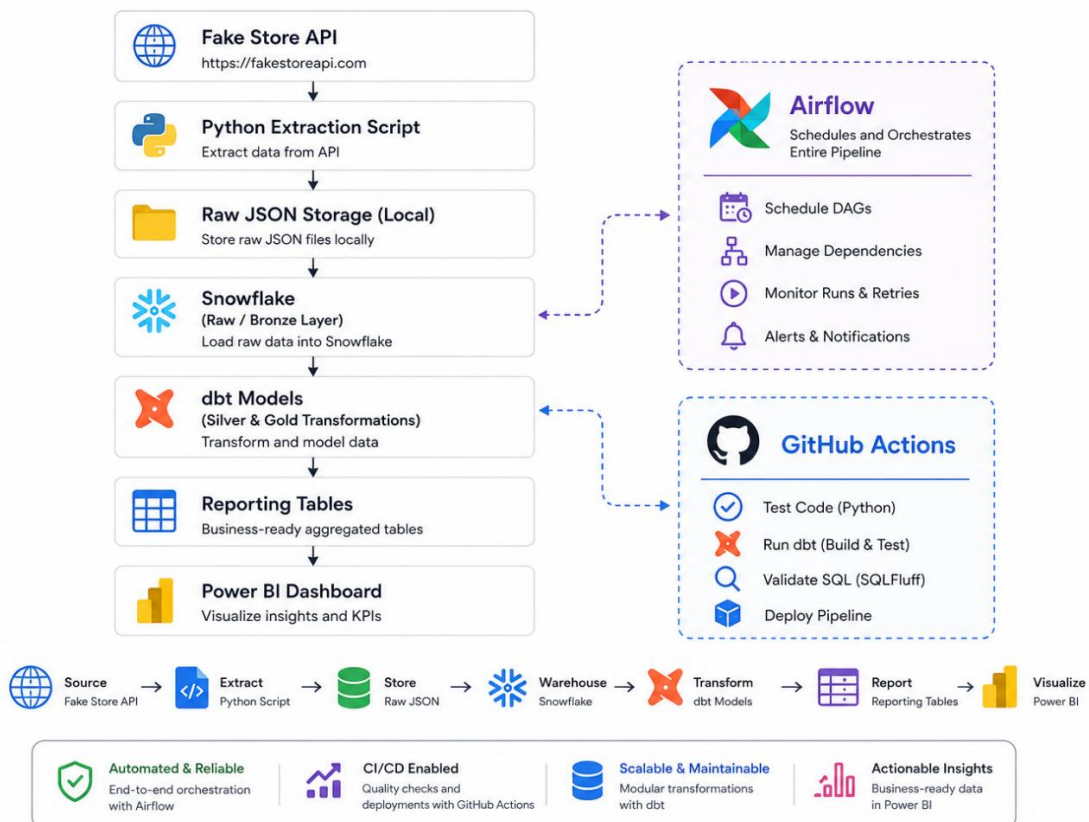
#### Project Objective

Students will build a complete modern data pipeline similar to those used by companies like Amazon, Netflix, Uber, and Airbnb.

The project demonstrates the entire data engineering lifecycle:

 API →  Raw Data →  Snowflake →  dbt →  Airflow →   
Reporting Tables →  Power BI →  GitHub Actions

#### Final Architecture



## Dataset

Use the Fake Store API.

API Endpoint

```
https://fakestoreapi.com/products
```

Example Record

```
{
  "id":1,
  "title":"Fjallraven Backpack",
  "price":109.95,
  "description":"...",
  "category":"men's clothing",
  "image":"...",
  "rating":{
    "rate":3.9,
    "count":120
  }
}
```

## Step 1 — Extract Data from API

### Goal

Extract product information using Python.

### Folder Structure

```
project/
|
├─ data/
|   raw/
|
├─ scripts/
|   extract.py
```

```
|  
├─ requirements.txt
```

## 🔧 Install Packages

```
pip install requests pandas
```

## 🐍 Python Script

```
import requests  
import pandas as pd  
  
url = "https://fakestoreapi.com/products"  
  
response = requests.get(url)  
  
products = response.json()  
  
df = pd.DataFrame(products)  
  
print(df.head())
```

## 💾 Save Raw Data

Create folder

```
data/raw/
```

Save JSON

```
import json  
  
with open("data/raw/products.json","w") as file:  
    json.dump(products,file,indent=4)
```

Save CSV

```
df.to_csv("data/raw/products.csv",index=False)
```

## ✓ Expected Output

```
products.json
```

```
products.csv
```

## 📁 Step 2 — Store Raw Data

### 🎯 Goal

Store raw files exactly as received.

Never modify raw data.

Raw Layer

```
Raw
```

```
|
```

```
├─ products.json
```

```
├─ products.csv
```

## Deliverable

✓ JSON

✓ CSV

✓ Folder Structure

## Step 3 — Load Data into Snowflake

### 🎯 Goal

Move raw data into Snowflake.

## Create Database

```
CREATE DATABASE ecommerce_db;
```

## Create Schema

```
CREATE SCHEMA raw;
```

## Create Table

```
CREATE TABLE raw.products (  
  id INTEGER,  
  title STRING,  
  price FLOAT,  
  description STRING,  
  category STRING,  
  image STRING,  
  rating VARIANT  
);
```

## Upload CSV

Use

Snowsight

Data

Load Data

Upload CSV

## Verify

```
SELECT *  
FROM raw.products;
```

## Deliverable

- ✓ Database
- ✓ Schema
- ✓ Table
- ✓ Loaded Data

## Step 4 — Transform Data using dbt

### Goal

Clean and model the data.

### Install dbt

```
pip install dbt-snowflake
```

### Initialize Project

```
dbt init ecommerce_project
```

### Model 1

```
models/  
stg_products.sql
```

```
SELECT  
  
id,  
  
title,  
  
price,  
  
category,  
  
rating:rate::FLOAT AS rating,  
  
rating:count::INTEGER AS reviews
```

```
FROM {{ source('raw', 'products') }}
```

## Run

```
dbt run
```

## Test

```
dbt test
```

## Deliverables

✓ dbt Project

✓ Models

✓ Tests

## Step 5 — Build Reporting Tables

### Goal

Create Gold Layer tables.

### Example 1

Average Price by Category

```
SELECT  
  
category,  
  
AVG(price) average_price,  
  
COUNT(*) total_products  
  
FROM stg_products  
  
GROUP BY category;
```



## Example 2

Highest Rated Products

```
SELECT *  
  
FROM stg_products  
  
ORDER BY rating DESC;
```

## Example 3

Price Categories

```
SELECT  
  
title,  
  
price,  
  
CASE  
  
WHEN price <50 THEN 'Budget'  
  
WHEN price <100 THEN 'Standard'  
  
ELSE 'Premium'  
  
END AS price_category  
  
FROM stg_products;
```

## Deliverables

Reporting Tables

```
gold.category_summary  
  
gold.product_ratings  
  
gold.price_segments
```

## Step 6 — Orchestrate using Airflow

### Goal

Automate the entire pipeline.

Pipeline Order

Extract [API](#)

↓

Save JSON

↓

Load Snowflake

↓

Run dbt

↓

Generate Reporting Tables

### DAG

daily\_pipeline.py

Tasks

extract\_api

↓

save\_raw

↓

load\_snowflake

↓

run\_dbt

↓

build\_reports

## Schedule

schedule="@daily"

## Deliverables

✓ DAG

✓ Successful Run



## Step 7 — Build Power BI Dashboard

## Connect

Power BI

↓

Snowflake

↓

Gold Tables

## Dashboard Pages



## Sales Overview

### Visuals

- Product Count
- Average Price
- Highest Price
- Lowest Price

---

## **Category Analysis**

### Visuals

- Bar Chart
- Tree Map
- Pie Chart

---

## **Ratings**

### Visuals

- Average Rating
- Top Rated Products
- Reviews

---

## **Pricing**

### Visuals

- Histogram
- Column Chart
- Scatter Plot

---

## **KPIs**

### Cards

- ✓ Total Products
- ✓ Average Rating

✓ Total Categories

✓ Average Price

---

## Deliverables

.pbix file

Dashboard screenshots

## Step 8 — GitHub Actions CI/CD

### Goal

Automatically validate the project whenever code is pushed.

Folder

```
.github  
workflows  
pipeline.yml
```

## Example Workflow

```
name: Data Pipeline CI  
  
on:  
  push:  
    branches:  
      - main  
  
jobs:  
  
  test:  
  
    runs-on: ubuntu-latest  
  
    steps:  
      - uses: actions/checkout@v4
```

```
- uses: actions/setup-python@v5

- run: pip install -r requirements.txt

- run: python scripts/extract.py

- run: dbt deps

- run: dbt run

- run: dbt test
```

## Deliverables

- ✓ GitHub Repository
- ✓ GitHub Actions Workflow
- ✓ Successful Pipeline

## Expected Project Structure

```
Capstone_Project/
|
├─ data/
|   raw/
|   processed/
|
├─ scripts/
|   extract.py
|   load_snowflake.py
|
├─ dbt_project/
|
```

```
├─ airflow/
|   dags/
|
├─ powerbi/
|   dashboard.pbix
|
├─ docs/
|   architecture.png
|
├─ .github/
|   workflows/
|       pipeline.yml
|
├─ requirements.txt
|
└─ README.md
```

## Learning Outcomes

By completing this capstone project, students will be able to:

-  Extract data from REST APIs using Python.
-  Store and manage raw datasets following data lake principles.
-  Load and query data efficiently in Snowflake.
-  Build modular, testable transformation pipelines with dbt.

- 🕒 Design and schedule end-to-end workflows using Apache Airflow.
- 📊 Create optimized reporting tables for analytics.
- 📈 Develop professional Power BI dashboards with actionable insights.
- 🚀 Implement CI/CD automation using GitHub Actions.
- 🏗️ Apply modern data engineering best practices, including layered architectures (Raw/Bronze, Silver, Gold), version control, testing, and documentation.